Lecture 3. Data Transformation

dplyr: filter, arrange, select, mutate, summarise, group_by, the pipe

Contents

3.1 Why transform data **2**
- 3.1.1 The five verbs . . . 2

3.2 Filter rows with filter() **2**
- 3.2.1 Comparisons . . . 3
- 3.2.2 Logical operators . . . 3
- 3.2.3 Missing values . . . 4

3.3 Arrange rows with arrange() **4**

3.4 Select columns with select() **5**

3.5 Add new variables with mutate() **5**
- 3.5.1 Useful creation functions . . . 5

3.6 Grouped summaries with summarise() **6**
- 3.6.1 The pipe . . . 7
- 3.6.2 Missing values and counts . . . 8
- 3.6.3 Useful summary functions . . . 8

3.7 Grouped mutates and filters **9**

3.1 Why transform data

Motivation. Data rarely arrive in exactly the form a plot or a model needs. We usually have to pick the relevant rows and columns, create new variables, reorder, or summarise. dplyr turns each of these into one verb, and chaining a few verbs answers most descriptive questions.

Visualisation generates insight, but the data first need new variables, summaries, renamed columns or reordered rows. This lecture uses `dplyr` (core tidyverse) and the `flights` data from `nycflights13`: all 336 776 flights that departed New York City in 2013.

```
library(nycflights13)
library(tidyverse)
```

```
flights
```

```
## # A tibble: 336,776 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     1      517            515         2      830            819        11
## 2  2013     1     1      533            529         4      850            830        20
## 3  2013     1     1      542            540         2      923            850        33
## 4  2013     1     1      544            545        -1     1004           1022       -18
## 5  2013     1     1      554            600        -6      812            837       -25
## # i 336,771 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

Under each column name the tibble shows its type: `int` (integer), `dbl` (double, real number), `chr` (character), `dttm` (date-time); later we also meet `lgl` (logical), `fctr` (factor) and `date`.

3.1.1 The five verbs

verb	action
`filter()`	pick observations (rows) by their values
`arrange()`	reorder rows
`select()`	pick variables (columns) by their names
`mutate()`	create new variables from existing ones
`summarise()`	collapse many values to a single summary

All of them work the same way: the first argument is a data frame, the next arguments say what to do using variable names without quotes, and the result is a new data frame. Combined with `group_by()`, which makes every verb act group by group, they form a small language for data manipulation.

3.2 Filter rows with filter()

Motivation. The first step of most questions is to restrict attention to the relevant cases. Doing this correctly needs care with comparisons of real numbers and with missing values.

```
jan1 <- filter(flights, month == 1, day == 1)
jan1
```

```
## # A tibble: 842 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     1      517            515         2      830            819        11
## 2  2013     1     1      533            529         4      850            830        20
```

```
## 3  2013     1     1      542            540         2      923            850        33
## 4  2013     1     1      544            545        -1     1004           1022       -18
## 5  2013     1     1      554            600        -6      812            837       -25
## # i 837 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

Arguments separated by commas are combined with "and". dplyr never modifies its input, so the result must be saved with `<-`; wrapping an assignment in parentheses, `(jan1 <- ...)`, also prints it.

3.2.1 Comparisons

The comparison operators are `>`, `>=`, `<`, `<=`, `!=` and `==`. The common mistake `filter(flights, month = 1)` uses assignment instead of equality; dplyr stops with an error that suggests `month == 1`.

Floating-point numbers are stored with finite precision, so every real number you see is an approximation:

```
sqrt(2) ^ 2 == 2
```

```
## [1] FALSE
```

```
1 / 49 * 49 == 1
```

```
## [1] FALSE
```

```
near(sqrt(2) ^ 2, 2)
```

```
## [1] TRUE
```

Use `near()`, which compares up to a small tolerance, instead of `==` for real numbers.

Why $1/49 \cdot 49 \neq 1$ [Supplement]

A double has a 53-bit binary mantissa, so $1/49$ is rounded to the nearest representable number. Multiplying back by 49 carries this rounding error, and the result differs from 1 in the last bit:

```
print(1 / 49 * 49, digits = 17)
```

```
## [1] 0.99999999999999989
```

`near(x, y)` returns `abs(x - y) < tol` with `tol` about 1.5×10^{-8} (the square root of machine epsilon).

3.2.2 Logical operators

`&` is "and", `|` is "or", `!` is "not"; `xor()` is exclusive or. Flights in November or December:

```
filter(flights, month == 11 | month == 12)
filter(flights, month %in% c(11, 12))      # the same, shorter
```

`filter(flights, month == 11 | 12)` does *not* work: it means `(month == 11) | 12`, and 12 is `TRUE`, so every flight is kept. De Morgan's laws, $!(x \,\&\, y) = !x \,|\, !y$ and $!(x \,|\, y) = !x \,\&\, !y$, give two ways of writing "delayed at most two hours both on arrival and departure":

```
filter(flights, !(arr_delay > 120 | dep_delay > 120))
filter(flights, arr_delay <= 120, dep_delay <= 120)
```

For complicated conditions, create them as explicit variables first so they can be checked. Do not use `&&` and `||` here: they handle only a single value and are meant for `if` statements (Lecture 10).

3.2.3 Missing values

`NA` ("not available") is an unknown value, and unknowns are contagious: almost any operation involving one is unknown.

```
NA > 5
```

```
## [1] NA
```

```
NA + 10
```

```
## [1] NA
```

```
NA == NA
```

```
## [1] NA
```

The last one makes sense with context: if Mary's age `x` and John's age `y` are both unknown, we cannot know whether they are the same age, so `x == y` is `NA`. To test for missingness use `is.na()`.

`filter()` keeps only rows where the condition is `TRUE`; it drops both `FALSE` and `NA`. To keep missing values, ask for them explicitly:

```
df <- tibble(x = c(1, NA, 3))
filter(df, x > 1)
```

```
## # A tibble: 1 x 1
##       x
##   <dbl>
## 1     3
```

```
filter(df, is.na(x) | x > 1)
```

```
## # A tibble: 2 x 1
##       x
##   <dbl>
## 1    NA
## 2     3
```

3.3 Arrange rows with arrange()

`arrange()` sorts rows by one or more columns, later columns breaking ties in earlier ones; `desc()` sorts in descending order. Missing values always go to the end, in either direction.

```
arrange(flights, desc(dep_delay))
```

```
## # A tibble: 336,776 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     9      641            900      1301     1242           1530      1272
## 2  2013     6    15     1432           1935      1137     1607           2120      1127
## 3  2013     1    10     1121           1635      1126     1239           1810      1109
## 4  2013     9    20     1139           1845      1014     1457           2210      1007
## 5  2013     7    22      845           1600      1005     1044           1815       989
## # i 336,771 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

```
df <- tibble(x = c(5, 2, NA))
arrange(df, desc(x))
```

```
## # A tibble: 3 x 1
##       x
##   <dbl>
## 1     5
## 2     2
## 3    NA
```

3.4 Select columns with select()

Motivation. Data sets with hundreds of variables are common, and the first job is often to narrow down to the few that matter.

```
select(flights, year, month, day)           # by name
select(flights, year:day)                   # all columns between year and day
select(flights, -(year:day))                # all except those
select(flights, seq(1, 10, by = 2))         # by position
select(flights, time_hour, air_time, everything())   # move to the front
rename(flights, tail_num = tailnum)         # rename, keep all others
```

Helper functions select by name pattern: `starts_with("abc")`, `ends_with("xyz")`, `contains("ijk")`, `matches("(.)\textbackslash \textbackslash 1")` (a regular expression, Lecture 7) and `num_range("x", 1:3)` for `x1`, `x2`, `x3`.

3.5 Add new variables with mutate()

Motivation. Many questions are about quantities that are not recorded directly, such as the speed of a flight or the time it made up in the air. mutate() computes them from existing columns.

`mutate()` adds columns at the end of the data set; to see them, start from a narrower data set:

```
flights_sml <- select(flights, year:day, ends_with("delay"), distance, air_time)
mutate(flights_sml,
  gain = dep_delay - arr_delay,
  hours = air_time / 60,
  gain_per_hour = gain / hours
)
```

```
## # A tibble: 336,776 x 10
##    year month   day dep_delay arr_delay distance air_time  gain hours gain_per_hour
##   <int> <int> <int>     <dbl>     <dbl>    <dbl>    <dbl> <dbl> <dbl>         <dbl>
## 1  2013     1     1         2        11     1400      227    -9  3.78         -2.38
## 2  2013     1     1         4        20     1416      227   -16  3.78         -4.23
## 3  2013     1     1         2        33     1089      160   -31  2.67        -11.6
## 4  2013     1     1        -1       -18     1576      183    17  3.05          5.57
## 5  2013     1     1        -6       -25      762      116    19  1.93          9.83
## # i 336,771 more rows
```

New columns can be used in the same call (`gain_per_hour` uses `gain`). `transmute()` keeps only the new variables.

3.5.1 Useful creation functions

The function must be **vectorised**: it takes a vector and returns a vector of the same length.

- Arithmetic `+ - * / ^`, with recycling of short arguments: `air_time / 60`, `x / sum(x)` (proportion), `y - mean(y)` (deviation).

- Modular arithmetic: `%/%` (integer division) and `%%` (remainder). Departure times are stored as `HHMM` numbers, so

```
transmute(flights, dep_time, hour = dep_time %/% 100, minute = dep_time %% 100)
```

```
## # A tibble: 336,776 x 3
##   dep_time  hour minute
##      <int> <dbl>  <dbl>
## 1      517     5     17
## 2      533     5     33
## 3      542     5     42
## 4      544     5     44
## 5      554     5     54
## # i 336,771 more rows
```

- Logs `log()`, `log2()`, `log10()`: useful for data over several orders of magnitude; `log2` is easiest to read (one unit = doubling).
- Offsets `lag()` and `lead()`, and cumulative aggregates `cumsum()`, `cumprod()`, `cummin()`, `cummax()`, `cummean()`:

```
x <- 1:10
lag(x)
```

```
##  [1] NA  1  2  3  4  5  6  7  8  9
```

```
cumsum(x)
```

```
##  [1]  1  3  6 10 15 21 28 36 45 55
```

```
cummean(x)
```

```
##  [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5
```

- Ranking: `min_rank()` gives ties the smallest rank (as in sports); `desc()` reverses. Variants: `row_number()`, `dense_rank()`, `percent_rank()`, `cume_dist()`, `ntile()`.

```
y <- c(1, 2, 2, NA, 3, 4)
min_rank(y)
```

```
## [1]  1  2  2 NA  4  5
```

```
min_rank(desc(y))
```

```
## [1]  5  3  3 NA  2  1
```

3.6 Grouped summaries with summarise()

Motivation. Summaries become informative when compared across groups: the average delay per day, per destination, per airline. group_by() plus summarise() is the workhorse for this.

`summarise()` collapses a data frame to one row:

```
summarise(flights, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   delay
##   <dbl>
## 1  12.6
```

It becomes useful together with `group_by()`, which changes the unit of analysis from the whole data set to groups. Grouping does not change how the data look, only how the other verbs act on them:

```
by_day <- group_by(flights, year, month, day)
summarise(by_day, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 365 x 4
## # Groups:   year, month [12]
##    year month   day delay
##   <int> <int> <int> <dbl>
## 1  2013     1     1 11.5
## 2  2013     1     2 13.9
## 3  2013     1     3 11.0
## 4  2013     1     4  8.95
## 5  2013     1     5  5.73
## # i 360 more rows
```

Each `summarise()` peels off one grouping level, which is what the message about grouped output means.

3.6.1 The pipe

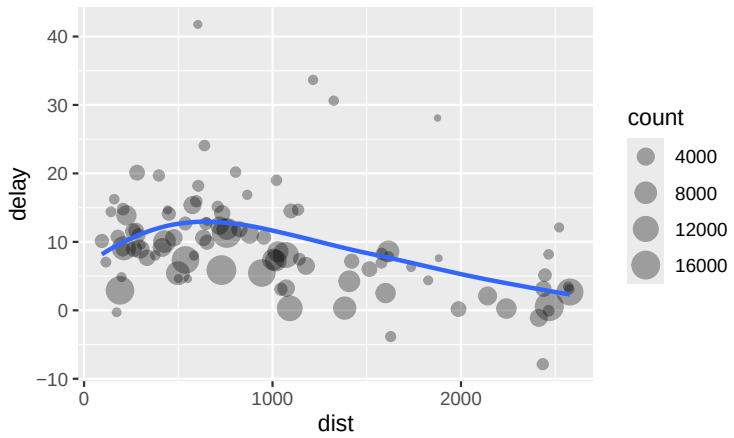
Relationship between distance and average delay for each destination, written with intermediate objects:

```
by_dest <- group_by(flights, dest)
delay <- summarise(by_dest, count = n(),
  dist = mean(distance, na.rm = TRUE), delay = mean(arr_delay, na.rm = TRUE))
delay <- filter(delay, count > 20, dest != "HNL")
```

Naming every intermediate result is tedious. The **pipe** `%>%` passes the result on the left as the first argument of the function on the right, so `x %>% f(y)` is `f(x, y)`; read it as "then":

```
delays <- flights %>%
  group_by(dest) %>%
  summarise(count = n(),
            dist = mean(distance, na.rm = TRUE),
            delay = mean(arr_delay, na.rm = TRUE)) %>%
  filter(count > 20, dest != "HNL")

ggplot(data = delays, mapping = aes(x = dist, y = delay)) +
  geom_point(aes(size = count), alpha = 1/3) +
  geom_smooth(se = FALSE)
```



Delays increase with distance up to about 750 miles and then decrease, perhaps because longer

flights can make up time in the air. ggplot2 layers are combined with `+`, not with the pipe (ggplot2 was written before the pipe).

3.6.2 Missing values and counts

Without `na.rm = TRUE` any missing value makes the summary `NA`. Missing delays here are cancelled flights, so it is cleaner to remove them once:

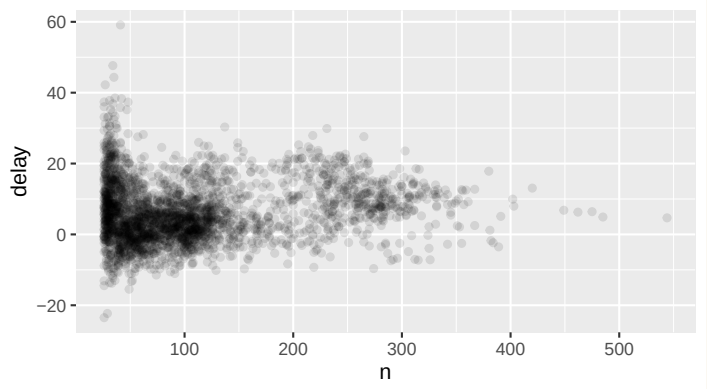
```
not_cancelled <- flights %>% filter(!is.na(dep_delay), !is.na(arr_delay))
```

Whenever you aggregate, include a count, `n()` or `sum(!is.na(x))`, so you do not draw conclusions from tiny groups: averages of few observations vary much more than averages of many.

Small groups have extreme averages [Supplement]

Plotting the average delay of each plane against its number of flights shows a funnel: planes with few flights have both the highest and the lowest averages. This is the variance σ^2/n of a mean at work, not a property of those planes. Filtering out small groups removes the noise:

```
not_cancelled %>%
  group_by(tailnum) %>%
  summarise(delay = mean(arr_delay), n = n()) %>%
  filter(n > 25) %>%
  ggplot(mapping = aes(x = n, y = delay)) +
  geom_point(alpha = 1/10)
```



3.6.3 Useful summary functions

- Location: `mean(x)`, `median(x)`; combined with logical subsetting, e.g. `mean(arr_delay[arr_delay > 0])` is the average positive delay.
- Spread: `sd(x)`, `IQR(x)`, `mad(x)`; the last two are robust to outliers.
- Rank: `min(x)`, `quantile(x, 0.25)`, `max(x)`.
- Position: `first(x)`, `nth(x, 2)`, `last(x)`.
- Counts: `n()`, `sum(!is.na(x))`, `n_distinct(x)`; `count(dest)` is a shortcut for grouping and counting, and `count(tailnum, wt = distance)` sums a weight (here total miles flown by each plane).
- Counts and proportions of logical values: `TRUE` counts as 1 and `FALSE` as 0, so `sum(x > 10)` is a count and `mean(y == 0)` a proportion.

```
not_cancelled %>%
  group_by(dest) %>%
  summarise(carriers = n_distinct(carrier)) %>%
  arrange(desc(carriers))
```

```
## # A tibble: 104 x 2
##   dest  carriers
##   <chr>    <int>
## 1 ATL          7
## 2 BOS          7
## 3 CLT          7
## 4 ORD          7
## 5 TPA          7
## # i 99 more rows
```

```
not_cancelled %>%
  group_by(year, month, day) %>%
  summarise(n_early = sum(dep_time < 500),      # flights before 5am
            hour_prop = mean(arr_delay > 60))   # share delayed over an hour
```

```
## # A tibble: 365 x 5
## # Groups:   year, month [12]
##    year month   day n_early hour_prop
##   <int> <int> <int>   <int>     <dbl>
## 1  2013     1     1       0    0.0722
## 2  2013     1     2       3    0.0851
## 3  2013     1     3       4    0.0567
## 4  2013     1     4       3    0.0396
## 5  2013     1     5       3    0.0349
## # i 360 more rows
```

Use `ungroup()` to remove grouping before operations that should act on the whole data set.

3.7 Grouped mutates and filters

Grouping also changes `filter()` and `mutate()`: functions such as `rank()`, `n()` and `mean()` are then computed within each group.

```
flights_sml %>%                          # the worst members of each group
  group_by(year, month, day) %>%
  filter(rank(desc(arr_delay)) < 10)
```

```
## # A tibble: 3,306 x 7
## # Groups:   year, month, day [365]
##    year month   day dep_delay arr_delay distance air_time
##   <int> <int> <int>     <dbl>     <dbl>    <dbl>    <dbl>
## 1  2013     1     1       853       851      184       41
## 2  2013     1     1       290       338     1134      213
## 3  2013     1     1       260       263      266       46
## 4  2013     1     1       157       174      213       60
## 5  2013     1     1       216       222      708      121
## # i 3,301 more rows
```

```
popular_dests <- flights %>%             # all groups bigger than a threshold
  group_by(dest) %>%
  filter(n() > 10000)
nrow(popular_dests)
```

```
## [1] 131440
```

```
popular_dests %>%                        # standardise to per-group metrics
  filter(arr_delay > 0) %>%
  mutate(prop_delay = arr_delay / sum(arr_delay)) %>%
  select(year:day, dest, arr_delay, prop_delay)
```

```
## # A tibble: 50,245 x 6
## # Groups:   dest [9]
##    year month   day dest  arr_delay prop_delay
##   <int> <int> <int> <chr>     <dbl>      <dbl>
## 1  2013     1     1 MIA          33  0.000235
## 2  2013     1     1 ORD          12  0.0000424
## 3  2013     1     1 FLL          19  0.0000938
## 4  2013     1     1 ORD           8  0.0000283
## 5  2013     1     1 LAX           7  0.0000344
## # i 50,240 more rows
```